Situation Calculus

Reasoning about action and agent programming

Sebastian Sardina
Department of Computer Science and Information Technology
RMIT University
Melbourne, AUSTRALIA

Jorge Baier
Christian Fritz
Alfredo Gabaldon
Hojjat Gadheri
Giuseppe De Giacomo
Eric I-Hung Hsu
Sam Kaufman
Todd Kelley
Iluji Kiringa
Yves Lespérance
Fangzhen Lin
Yongmei Liu

www.cs.toronto.edu/cogrobo

Ray Reiter
Hector Levesque

Daniel Marcu
Sheila McIlraith
Maurice Pagnucco
Ron Petrick
Javier Pinto
Shane Ruman
Sebastian Sardina
Richard Scherl
Steven Shapiro
Mikhail Soutchanski
Eugenia Ternovskaia
Stavros Vassos

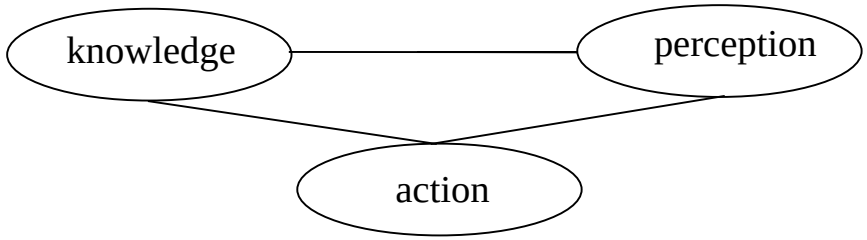
Outline

Disclaimer: some slides are based on were "stolen" from Hector Levesque, Giuseppe de Giacomo, and Ryan Kelly, and others. :-)

Outline

Cognitive Robotics

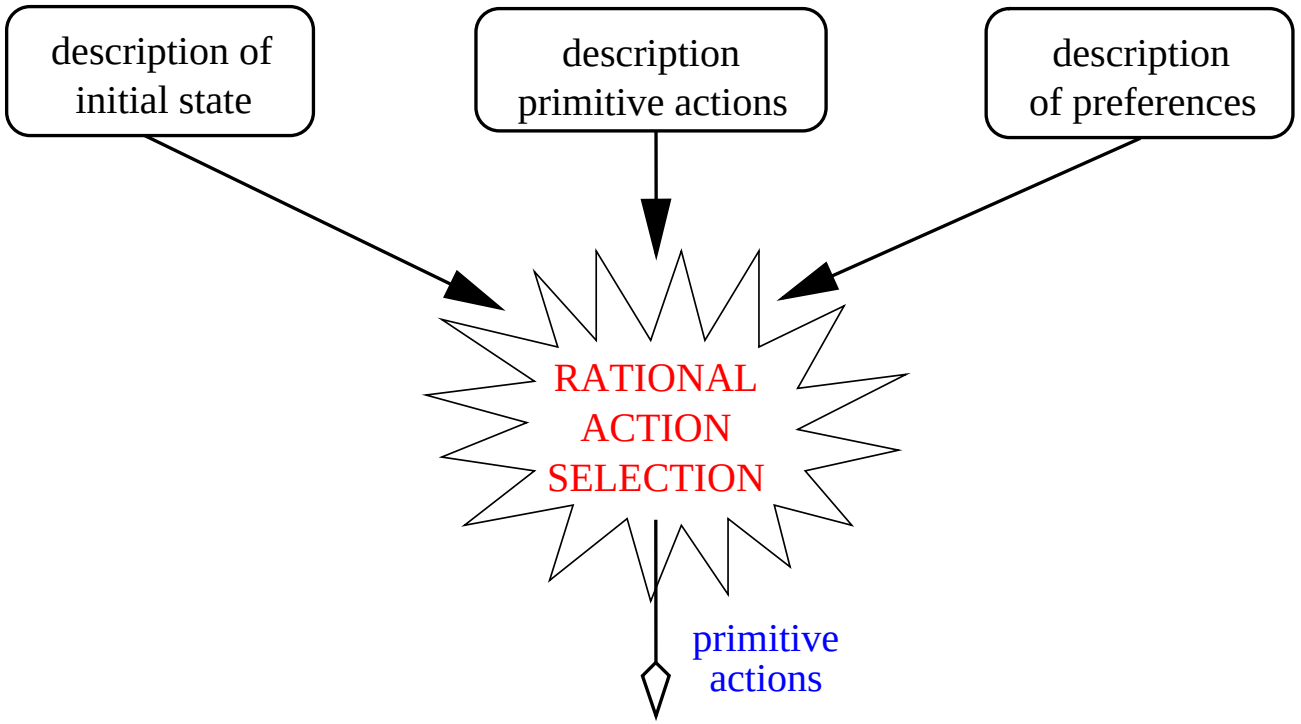
The study of the knowledge representation and reasoning problems faced by an autonomous agent in a dynamic and incompletely known world.



- knowledge of the agent vs. the designer?
- perception vs. reasoning?
- when is initial knowledge adequate to achieve a goal?

Do not want to simply engineer robot controllers that solve a class of problems or that work in a class of application domains.

Doing the right thing

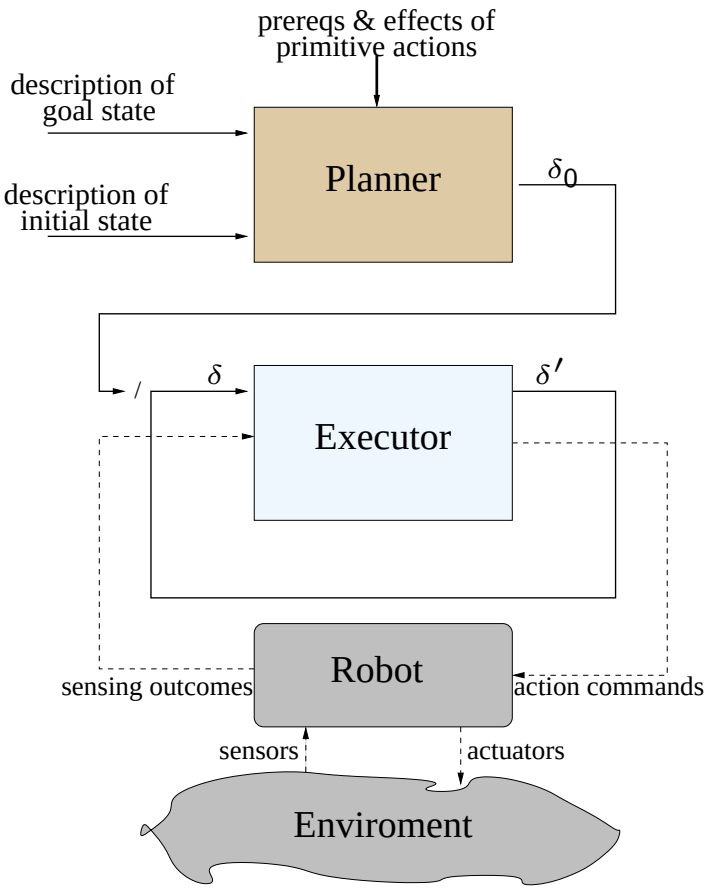


would like a system that is capable of generating actions in the world that are appropriate, as a function of some current set of beliefs and desires.

Note: the descriptions and preferences are inputs (not wired in!)

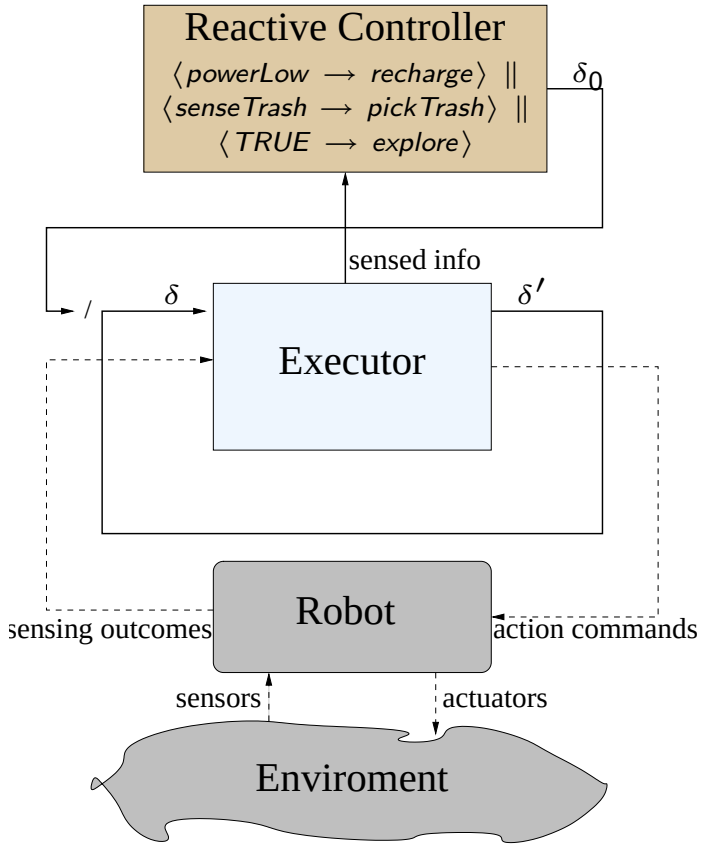
Classical Planning [SRI Shakey "The Robot" '70]

- Planing from first-principle:
  Goal oriented (usually goals-to-be)
- Dumb executor, little feedback,
  no reactivity, does not scale!
- Kind of plans δ_0:
  - Linear total-order plans (STRIPS)
  - Partial-order plans (SNLP)
  - Conditional plans (CNLP, S-GP)
  - **Robot programs** [Levesque 96]
- An interesting case: [Sacerdoti 74]
  HTN planning/programming



The Reactive Approach [Brooks 86,91]

- Reactive robots, headless!
- Little representation, quick decisions
- But: little or no pro-activity
- Examples:
  - Subsumption architecture [Brooks 86]
  - Universal plans [Schoopers 87]
  - Control Reactive Rules [Baral&Son 98]
  - (PO)MDP and policies?



AGENT = Action Theory + High-Level Program

In *Cognitive Robotics*, we think of an agent equipped with:

- An action theory: the agent knows the theory and its consequences (actions' effects, frame & qualification problems, sensing, etc.)
- A high-level program: specifying the agent tasks/behaviors (nondeterministic & domain actions)

AGENT = Action Theory + High-Level Program

In *Cognitive Robotics*, we think of an agent equipped with:

- An action theory: the agent knows the theory and its consequences (actions' effects, frame & qualification problems, sensing, etc.)
- A high-level program: specifying the agent tasks/behaviors (nondeterministic & domain actions)

$$
\begin{aligned}
\mathit{getOnFlight} \stackrel{\text{def}}{=}\ & (\pi a.a)^*; \mathit{At}(\mathit{airport})?; \\
& (\mathit{go}(\mathit{terminal}1) \mid \mathit{go}(\mathit{terminal}2)); \\
& (\mathit{buy}(\mathit{magazine}) \mid \mathit{buy}(\mathit{newspaper})); \\
& \textbf{if } (\mathit{GateNo} \geq 90) \textbf{ then } \{\mathit{go}(\mathit{gate}(\mathit{GateNo})); \mathit{buy}(\mathit{coffee})\} \\
& \qquad\qquad \textbf{else } \{\mathit{buy}(\mathit{coffee}); \mathit{go}(\mathit{gate}(\mathit{GateNo}))\} \\
& \mathit{boardPlane}
\end{aligned}
$$

High-Level Programs

Programs telling the robot what needs to be done:

- primitive statements are actions to be performed by the robot:
  $\Longrightarrow$ *move to the desk and then pick up the package*
- tests within a program pertain to conditions in the world that are affected by the actions of the robot (or other agents):
  $\Longrightarrow$ **if** *the door is locked* **then** ... **otherwise** ...
  $\Longrightarrow$ **while** *there is a package on the table* **do** ...
- programs may be non-deterministic: they may contain choice points where the interpreter must make a reasoned (non-random) selection:
  $\Longrightarrow$ *go through the (appropriate) door and retrieve the package that is waiting*
  $\Longrightarrow$ *either turn left or turn right (as appropriate) ... at which point you must be located in the hall*

High-Level Programs

Programs telling the robot what needs to be done:

- primitive statements are actions to be performed by the robot:
  $\Longrightarrow$ *move to the desk and then pick up the package*
- tests within a program pertain to conditions in the world that are affected by the actions of the robot (or other agents):
  $\Longrightarrow$ **if** *the door is locked* **then** *...* **otherwise** *...*
  $\Longrightarrow$ **while** *there is a package on the table* **do** *...*
- programs may be non-deterministic: they may contain choice points where the interpreter must make a reasoned (non-random) selection:
  $\Longrightarrow$ *go through the (appropriate) door and retrieve the package that is waiting*
  $\Longrightarrow$ *either turn left or turn right (as appropriate) ... at which point you must be located in the hall*

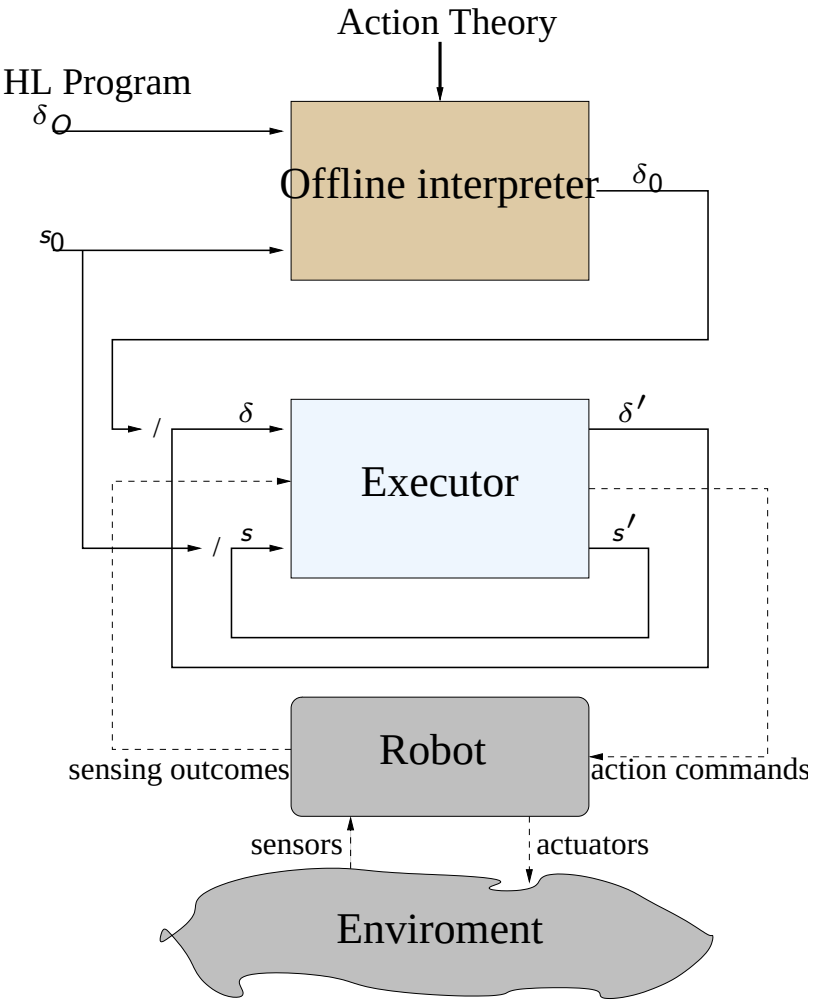
High-Level Programs

Programs telling the robot what needs to be done:

- primitive statements are actions to be performed by the robot:
  $\Longrightarrow$ *move to the desk and then pick up the package*
- tests within a program pertain to conditions in the world that are affected by the actions of the robot (or other agents):
  $\Longrightarrow$ **if** *the door is locked* **then** ... **otherwise** ...
  $\Longrightarrow$ **while** *there is a package on the table* **do** ...
- programs may be non-deterministic: they may contain choice points where the interpreter must make a reasoned (non-random) selection:
  $\Longrightarrow$ *go through the (appropriate) door and retrieve the package that is waiting*
  $\Longrightarrow$ *either turn left or turn right (as appropriate) ... at which point you must be located in the hall*

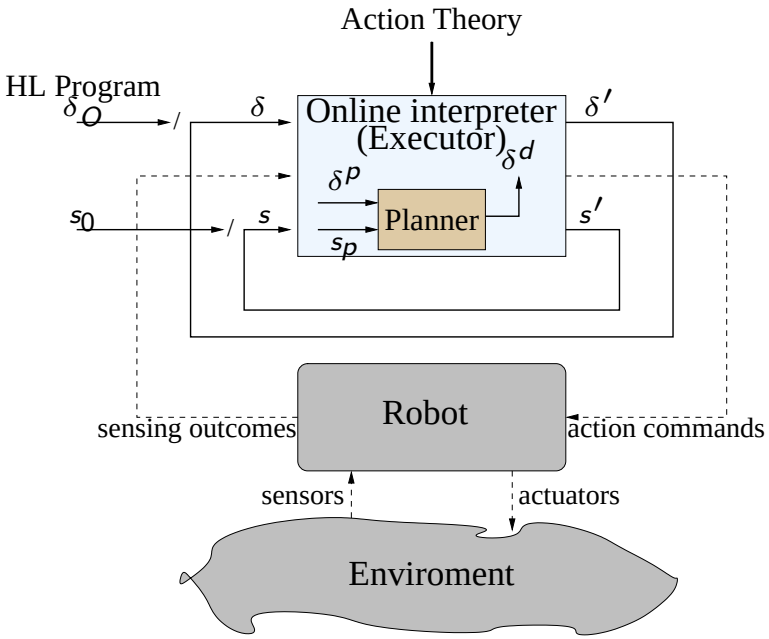
The Agent Programming Approach I

- Planning with *procedural* clues
  Goals-to-do
- Less complexity! But still hard!
- Little reactive and feedback.
- Examples:
  - GOLOG, CONGOLOG, DTGOLOG
  - MDPs + HAM [Parr 98]
  - HTN-planning [Eral et. al 94]



The Agent Programming Architecture II [INDIGOLOG]

- High-level programs executed online or incrementally
- Planning is just a *sub-module*
- Control over extended periods
- Reactive: can adapt
- Proactive: plan locally
- Can handle runtime sensing



We will talk about...

1. Action theories in the situation calculus.
2. Offline execution of high-level programs.
3. Online execution of high-level programs and local planning.

Outline

Situation Calculus [McCarthy and Hayes 69]

The description of the initial state, and the prerequisites and effects of actions can be formulated in the language of the *situation calculus*:

1. A dialect of the predicate calculus with special sorts for:
   - actions: $closeDoor$, $pickUp(package\_13)$, $moveFwd(2)$;
   - situations: S_0, the initial situation, $do(a, s)$, where a is an action term and s is a situation term.
2. Predicates or functions whose value changes from situation to situation are called fluents:

$$\neg Holding(x, \underline{S_0}) \wedge Holding(x, \underline{do(pickUp(x), S_0)})$$

3. Use a distinguished predicate $Poss(a, s)$ to assert that action a is possible to execute in situation s.

Basic Action Theories [Lin and Reiter 94]

Theory $\mathcal{D} = \Sigma \cup \mathcal{D}_{S_0} \cup \mathcal{D}_{Poss} \cup \mathcal{D}_{SSA} \cup \mathcal{D}_{UNA}$, where:

- $\mathcal{D}_{S_0}$: axioms describing the initial state of the fluents in S_0;
  $\Longrightarrow$ allows for incomplete knowledge!
- $\mathcal{D}_{poss}$: axioms describing the prerequisites of the primitive actions, of the form:
$$Poss(A(\vec{x}), s) \equiv \Theta_A(\vec{x}, s),$$
one for each primitive action A.
- $\mathcal{D}_{ssa}$: axioms describing the effects and non-effects of actions, of the form:
$$F(\vec{x}, do(a, s)) \equiv \psi_F(\vec{x}, a, s),$$
one for each fluent F.
- $\mathcal{D}_{una}$: unique name axioms for the actions.
- Σ: some foundational domain-independent axioms.

A solution to the frame problem (sometimes) [Reiter 91]

1. Positive effect axioms:

$$Holding(x, s) \wedge Fragile(x, s) \supset Broken(x, \underline{do(drop(x), s)})$$

2. Positive effect axioms:

$$HasGlue(s) \wedge Broken(x, s) \supset \neg Broken(x, \underline{do(repair(x), s)})$$

3. Frame axiom axioms:

$$\neg Broken(x, s) \wedge (y \neq x \vee \neg Fragile(x, s)) \supset \neg Broken(x, \underline{do(drop(y), s)})$$

4. Completeness assumption:

$$\neg Broken(x, s) \wedge Broken(x, do(a, s)) \supset Fragile(x, s) \wedge a = drop(x)$$

Successor state axioms = positive effect axioms + positive effect axioms + frame axioms + completness assumption

A solution to the frame problem (sometimes) [Reiter 91]

So, suppose that for fluent $F(\vec{x}, s)$:

$\gamma_F^+(\vec{x}, a, s)$ encodes all *positive* effects; and
$\gamma_F^-(\vec{x}, a, s)$ encodes all *negative* effects.

The successor state axiom solution has the following form:

$$F(\vec{x}, do(a, s)) \equiv \gamma_F^+(\vec{x}, a, s) \vee F(\vec{x}, s) \wedge \neg\gamma_F^-(\vec{x}, a, s)$$

In our example:

A solution to the frame problem (sometimes) [Reiter 91]

So, suppose that for fluent $F(\vec{x}, s)$:

$\gamma_F^+(\vec{x}, a, s)$ encodes all *positive* effects; and
$\gamma_F^-(\vec{x}, a, s)$ encodes all *negative* effects.

The successor state axiom solution has the following form:

$$F(\vec{x}, do(a, s)) \equiv \gamma_F^+(\vec{x}, a, s) \vee F(\vec{x}, s) \wedge \neg\gamma_F^-(\vec{x}, a, s)$$

In our example:

$$\gamma_{Broken}^+(\vec{x}, a, s) \stackrel{\text{def}}{=} \ldots$$
$$\gamma_{Broken}^-(\vec{x}, a, s) \stackrel{\text{def}}{=} \ldots$$

$$Broken(x, do(a, s)) \equiv \gamma_{Broken}^+(\vec{x}, a, s) \vee Broken(\vec{x}, s) \wedge \neg\gamma_{Broken}^-(\vec{x}, a, s)$$

A solution to the frame problem (sometimes) [Reiter 91]

So, suppose that for fluent $F(\vec{x}, s)$:

$\gamma_F^+(\vec{x}, a, s)$ encodes all *positive* effects; and
$\gamma_F^-(\vec{x}, a, s)$ encodes all *negative* effects.

The successor state axiom solution has the following form:

$$F(\vec{x}, do(a, s)) \equiv \gamma_F^+(\vec{x}, a, s) \vee F(\vec{x}, s) \wedge \neg\gamma_F^-(\vec{x}, a, s)$$

In our example:

$$\gamma_{Broken}^+(\vec{x}, a, s) \stackrel{\text{def}}{=} a = drop(x) \wedge Holding(x, s) \wedge Fragile(x, s)$$
$$\gamma_{Broken}^-(\vec{x}, a, s) \stackrel{\text{def}}{=} \ldots$$

$$Broken(x, do(a, s)) \equiv [Holding(x, s) \wedge Fragile(x, s) \wedge a = drop(x)] \vee Broken(x, s) \wedge \neg\gamma_{Broken}^-(\vec{x}, a, s)$$

A solution to the frame problem (sometimes) [Reiter 91]

So, suppose that for fluent $F(\vec{x}, s)$:

$\gamma_F^+(\vec{x}, a, s)$ encodes all *positive* effects; and
$\gamma_F^-(\vec{x}, a, s)$ encodes all *negative* effects.

The successor state axiom solution has the following form:

$$F(\vec{x}, do(a, s)) \equiv \gamma_F^+(\vec{x}, a, s) \vee F(\vec{x}, s) \wedge \neg\gamma_F^-(\vec{x}, a, s)$$

In our example:

$$\gamma_{Broken}^+(\vec{x}, a, s) \stackrel{\text{def}}{=} a = drop(x) \wedge Holding(x, s) \wedge Fragile(x, s)$$
$$\gamma_{Broken}^-(\vec{x}, a, s) \stackrel{\text{def}}{=} a = repair(x) \wedge HasGlue(x, s)$$

$$Broken(x, do(a, s)) \equiv$$
$$[Holding(x, s) \wedge Fragile(x, s) \wedge a = drop(x)] \vee$$
$$Broken(x, s) \wedge \neg[a = repair(x) \wedge HasGlue(s)]$$

Reasoning about actions

- Projection task: *given a sequence of actions* $a_1, \ldots, a_n$, *does condition* $\phi(s)$ *holds?*:

$$\mathcal{D} \models \phi(do(do(a_n, do(a_{n-1}, \ldots, do(a_1, S_0)) \ldots)$$

- Deductive planning: *find a sequence of actions to make* $\phi(s)$ *true*:

$$\mathcal{D} \vdash \exists s.\phi(s) \wedge executable(s)$$

- Database transaction formalization:

$$\mathcal{D} \models \exists c.Enrolled(john, c, do(reg(mary, c10), do(drop(john, c21), S_0)))$$

- Inductive proofs:

$$\mathcal{D} \models \forall p, n, n'.s \sqsubseteq s' \wedge sal(p, n, s) \wedge sal(p, n', s') \supset n \leq n'$$

Reasoning about actions (cont.)

- Knowledge and beliefs: what does *the agent* knows?

$$\phi(s) \text{ vs } \mathbf{Know}(\phi, s).$$

- Explicit time and continuous change:

$$start(s) = t \text{ and } Velocity(x, s) = 87.$$

- Natural actions: actions that *must* occur (bouncing of the ball).
- Concurrent actions:

$$\mathcal{D} \models \neg Spilled(coffee, do(\{liftRight(table), liftLeft(table)\}, S_0),$$
$$\mathcal{D} \models Spilled(coffee, do(\{liftRight(table)\}, S_0).$$

- Ramifications: indirect effects

$$\mathcal{D} \models \forall s. Inside(x, y) \supset At(x, l, do(move(y, l), S_0)).$$

Reasoning about actions (cont.)

- Knowledge and beliefs: what does *the agent* knows?

$$\phi(s) \text{ vs } \mathbf{Know}(\phi, s).$$

- Explicit time and continuous change:

$$start(s) = t \text{ and } Velocity(x, s) = 87.$$

- Natural actions: actions that *must* occur (bouncing of the ball).
- Concurrent actions:

$$\mathcal{D} \models \neg Spilled(coffee, do(\{liftRight(table), liftLeft(table)\}, S_0),$$
$$\mathcal{D} \models Spilled(coffee, do(\{liftRight(table)\}, S_0).$$

- Ramifications: indirect effects

$$\mathcal{D} \models \forall s. Inside(x, y) \supset At(x, l, do(move(y, l), S_0)).$$

All in plain classical (i.e., predicate) logic!

Outline

GOLOG [Levesque 96]

GOLOG is a *logic-based* programming language whose primitive actions are those of a background action theory.

It includes the following constructs:

α,	*primitive action*
$\phi?$,	*test/wait for a condition*
$(\delta_1; \delta_2)$,	*sequence*
if ϕ **then** δ_1 **else** δ_2 **endIf**,	*conditional*
while ϕ **do** δ **endWhile**,	*loop*
proc $\beta(\vec{x})$ δ **endProc**,	*procedure definition*
$\beta(\vec{t})$,	*procedure call*
$(\delta_1 \mid \delta_2)$,	*nondeterministic choice of action*
$\pi\, \vec{x}\, [\delta]$,	*nondet. choice of arguments*
δ^*,	*nondeterministic iteration*

GOLOG semantics

An offline execution of a GOLOG program δ relative to an action theory $\mathcal{D}$, is a situation term S such that:

$$\mathcal{D} \models Do(\delta, S_0, S).$$

$Do(\delta, s, s')$: *it is possible to reach situation* s' *from situation* s *by executing a sequence of actions specified by* δ.

GOLOG semantics

An offline execution of a GOLOG program δ relative to an action theory $\mathcal{D}$, is a situation term S such that:

$$\mathcal{D} \models Do(\delta, S_0, S).$$

$Do(\delta, s, s')$: *it is possible to reach situation s' from situation s by executing a sequence of actions specified by δ.*

This execution formula can be defined inductively:

$$\begin{aligned}
Do(a, s, s') &\stackrel{\text{def}}{=} Poss(a, s) \wedge s' = do(a, s) \\
Do(?\phi, s, s') &\stackrel{\text{def}}{=} \phi(s) \wedge s' = s \\
Do(\delta_1; \delta_2, s, s') &\stackrel{\text{def}}{=} \exists s''. Do(\delta_1, s, s'') \wedge Do(\delta_2, s'', s') \\
Do(\delta_1 \mid \delta_2, s, s') &\stackrel{\text{def}}{=} Do(\delta_1, s, s') \vee Do(\delta_2, s, s') \\
Do(\pi x.\delta(x), s, s') &\stackrel{\text{def}}{=} \exists x. Do(\delta(x), s, s')
\end{aligned}$$

GOLOG implementation

Do can be *lifted* directly from its semantic specification:

```
do([E1,E2],S,S1) :- do(E1,S,S2), do(E2,S2,S1).
do(?(P),S,S) :- holds(P,S).
do(ndet(E1,E2),S,S1) :- do(E1,S,S1) ; do(E2,S,S1).
do(if(P,E1,E2),S,S1) :- do(ndet([?(P),E1],[?(-P),E2]),S,S1).
do(star(E),S,S1) :- S1 = S ; do([E,star(E)],S,S1).
do(while(P,E),S,S1):- do([star([?(P),E]),?(-P)],S,S1).
do(pi(V,E),S,S1) :- sub(V,_,E,E1), do(E1,S,S1).
do(E,S,S1) :- proc(E,E1), do(E1,S,S1).
do(E,S,do(E,S)) :- prim_action(E), poss(E,P), holds(P,S)

holds(and(P,Q),S) :- holds(P,S), holds(Q,S).
...
```

Some GOLOG applications

A number of applications have been written in GOLOG and variants:

- a simulated elevator controller;
- mail delivery robots at Toronto and York Universities
  $\Longrightarrow$ demonstrates hardware independence
- a robot museum guide [Burgard et al AAAI 98]
  $\Longrightarrow$ controllable online via the Web
- business process simulation [Yu & Mylopoulos 97]
  $\Longrightarrow$ for analysis of the processes, not execution
- characters in computer animation [Funge SIGGRAPH 99]
  $\Longrightarrow$ for high-level behaviour specification
- a large "softbot" application [Ruman MSc 96]
  personal banking: ATM controller, personal agents, bank agents, routers agents run as processes under Unix, communicate over TCP/IP

Why is Golog popular?

[Copied from Ryan Kelly's presentation, 2005]

Why is Golog popular?

[Copied from Ryan Kelly's presentation, 2005]

- Good level of abstraction:
  - Programs based directly on actions from the domain.
  - Nondeterminism makes programs simpler and more powerful.
  - Symbolic reasoning effortlessly available.

Why is Golog popular?

[Copied from Ryan Kelly's presentation, 2005]

- Good level of abstraction:
  - Programs based directly on actions from the domain.
  - Nondeterminism makes programs simpler and more powerful.
  - Symbolic reasoning effortlessly available.
- Tradeoff between programming and planning:
  - Amount of nondeterminism controlled by the programmer.
  - Procedural knowledge easy to encode.
  - Full planning still available.

Why is Golog popular?

[Copied from Ryan Kelly's presentation, 2005]

- Good level of abstraction:
  - Programs based directly on actions from the domain.
  - Nondeterminism makes programs simpler and more powerful.
  - Symbolic reasoning effortlessly available.
- Tradeoff between programming and planning:
  - Amount of nondeterminism controlled by the programmer.
  - Procedural knowledge easy to encode.
  - Full planning still available.
- Logic-based:
  - Compact implementation in Prolog.
  - Possible to prove safety/liveness properties.

Outline

What is missing?

A number of concerns are not well addressed by these efforts:

1. sensing and exogenous actions
2. reactivity: suspending some preprogrammed behaviour
   $\Longrightarrow$ e.g., monitoring power consumption
3. reasoned failure recovery
   $\Longrightarrow$ e.g., encountering an unexpected closed door
4. reliable and non-inmediate primitive actions
   $\Longrightarrow$ e.g., $go\_to(loc)$ **vs.** $start\_going\_to(loc)$ + exog. reports
5. local decision making
   $\Longrightarrow$ e.g., searching through an entire execution **vs.** committing quickly to the first action

These issues are addressed in more recent dialects of GOLOG.

Reactivity in CONGOLOG [de Giacomo et al AIJ 00]

An extension to GOLOG, which includes concurrency, prioritized interrupts, and exogenous actions.

$$\begin{array}{rcl}
\langle\ \mathit{TooHot} \wedge \neg \mathit{FanOn} & \rightarrow & \mathit{toggle_fan}\ \rangle\rangle \\
\langle\ \mathit{TooCold} \wedge \mathit{FanOn} & \rightarrow & \mathit{toggle_fan}\ \rangle\ \rangle \\
\langle\ \mathit{SmokeDetectorOn} & \rightarrow & \mathit{ring_alarm}\ \rangle\ \rangle \\
\langle\ n\ :\ \mathit{ButtonOn}(n) & \rightarrow & \pi f.?(\mathit{BestButton}(f)); \mathit{serve}(f)\rangle\ \rangle \\
\langle\ \neg \mathit{OnFloor}(1) & \rightarrow & \mathit{down}\rangle
\end{array}$$

Allows reactive controllers, but *still via reasoning about actions*!

- $\delta_1 \| \delta_2$: concurrent execution (equal priority)
- $\delta_1 \rangle\rangle \delta_2$: concurrent execution ($\delta_1$ with higher priority)
- $\delta^{\|}$: concurrent iteration
- $\langle\ \vec{x} : \phi(\vec{x}) \rightarrow \delta(\vec{x})\ \rangle$: interrupt

Semantics of CONGOLOG: *Trans* and *Final*

To characterize CONGOLOG behaviour, we need a theory of complex actions based on single steps:

1. $Trans(\delta, s, \delta', s')$: program δ can legally execute a single step to s', with δ' remaining to execute.
2. $Final(\delta, s)$: program δ can legally terminate.

$$Trans(a, s, \delta', s') \equiv Poss(a, s) \wedge s' = s \wedge \delta' = nil$$

$$Trans(\delta_1 \| \delta_2, s, \delta', s') \equiv \exists \delta''. Trans(\delta_1, s, \delta'', s') \wedge \delta' = (\delta''; \delta_2) \vee Trans(\delta_2, s, \delta'', s') \wedge \delta' = (\delta_1; \delta'')$$

$$Final(\textbf{while } \phi \textbf{ do } \delta, s) \equiv \neg\phi(s)$$

$$Final(\delta^*, s) \equiv TRUE$$

Semantics of CONGOLOG: *Trans* and *Final*

To characterize CONGOLOG behaviour, we need a theory of complex actions based on single steps:

1. $Trans(\delta, s, \delta', s')$: program δ can legally execute a single step to s', with δ' remaining to execute.
2. $Final(\delta, s)$: program δ can legally terminate.

$$Trans(a, s, \delta', s') \equiv Poss(a, s) \wedge s' = s \wedge \delta' = nil$$

$$Trans(\delta_1 \| \delta_2, s, \delta', s') \equiv \exists \delta''. Trans(\delta_1, s, \delta'', s') \wedge \delta' = (\delta''; \delta_2) \vee Trans(\delta_2, s, \delta'', s') \wedge \delta' = (\delta_1; \delta'')$$

$$Final(\textbf{while } \phi \textbf{ do } \delta, s) \equiv \neg\phi(s)$$

$$Final(\delta^*, s) \equiv TRUE$$

$$\mathcal{D} \cup \mathcal{C} \models \exists s'. Do(\delta, s, s'),$$

where $Do(\delta, s, s') \stackrel{\text{def}}{=} \exists \delta'. Trans^*(\delta, s, \delta', s') \wedge Final(\delta', s')$

Semantics of CONGOLOG: *Trans* and *Final*

To characterize CONGOLOG behaviour, we need a theory of complex actions based on single steps:

1. $Trans(\delta, s, \delta', s')$: program δ can legally execute a single step to s', with δ' remaining to execute.
2. $Final(\delta, s)$: program δ can legally terminate.

$$Trans(a, s, \delta', s') \equiv Poss(a, s) \wedge s' = s \wedge \delta' = nil$$

$$Trans(\delta_1 \| \delta_2, s, \delta', s') \equiv \exists \delta''. Trans(\delta_1, s, \delta'', s') \wedge \delta' = (\delta''; \delta_2) \vee Trans(\delta_2, s, \delta'', s') \wedge \delta' = (\delta_1; \delta'')$$

$$Final(\textbf{while } \phi \textbf{ do } \delta, s) \equiv \neg\phi(s)$$

$$Final(\delta^*, s) \equiv TRUE$$

$\mathcal{D} \cup \mathcal{C} \models \exists s'. Do(\delta, s, s')$, ***** STILL OFFLINE! *****

where $Do(\delta, s, s') \stackrel{\text{def}}{=} \exists \delta'. Trans^*(\delta, s, \delta', s') \wedge Final(\delta', s')$

Outline

INDIGOLOG [Sardina 2000,2004]

To allow sensing or exogenous inputs to help determine what action to perform next, we must execute programs online/incrementally!

We cannot compute an entire execution of a program offline before starting. (This was impractical for large programs anyway!)

Account based on *Trans* and *Final* is well-suited to online exec.:

INDIGOLOG [Sardina 2000,2004]

To allow sensing or exogenous inputs to help determine what action to perform next, we must execute programs online/incrementally!

We cannot compute an entire execution of a program offline before starting. (This was impractical for large programs anyway!)

Account based on *Trans* and *Final* is well-suited to online exec.:

> *find an action A such that* $Trans(\delta, s, \delta', do(A, s))$ *holds, and then commit to it (physically execute it). Repeat.*

INDIGOLOG [Sardina 2000,2004]

To allow sensing or exogenous inputs to help determine what action to perform next, we must execute programs online/incrementally!

We cannot compute an entire execution of a program offline before starting. (This was impractical for large programs anyway!)

Account based on *Trans* and *Final* is well-suited to online exec.:

> *find an action A such that* $Trans(\delta, s, \delta', do(A, s))$ *holds, and then commit to it (physically execute it). Repeat.*

Top level interpreter for INDIGOLOG:

```
indigo(P,S):- exog_occurs(A), !, indigo(P,[A|S]).
indigo(P,S):- final(P,S), !.
indigo(P,S):- trans(P,S,P1,S), !, indigo(P1,S).
indigo(P,S):- trans(P,S,P1,[A|S]),exec(A,R),indigo(P1,[A|S])
```

Local search $\Sigma(\delta)$

Unlike ordinary GOLOG, INDIGOLOG handles non-determinism by making a random selection:

> *The non-deterministic program* $(\delta_1 \mid \delta_2); \ldots; p?$ *may fail if the wrong action is selected and executed!*

The solution: extend the language with a new operator Σ where $\Sigma(\delta)$ means "take transitions in δ that will ultimately succeed":

$$Trans(\Sigma(\delta), s, \delta', s') \equiv Trans(\delta, s, \delta', s') \wedge (\exists s'')Do(\delta', s', s'').$$

Can control the amount of lookahead search:

$$\Sigma(\delta_1); \delta_2 \quad \textbf{vs} \quad \Sigma(\delta_1; \delta_2)$$

An example of local planning

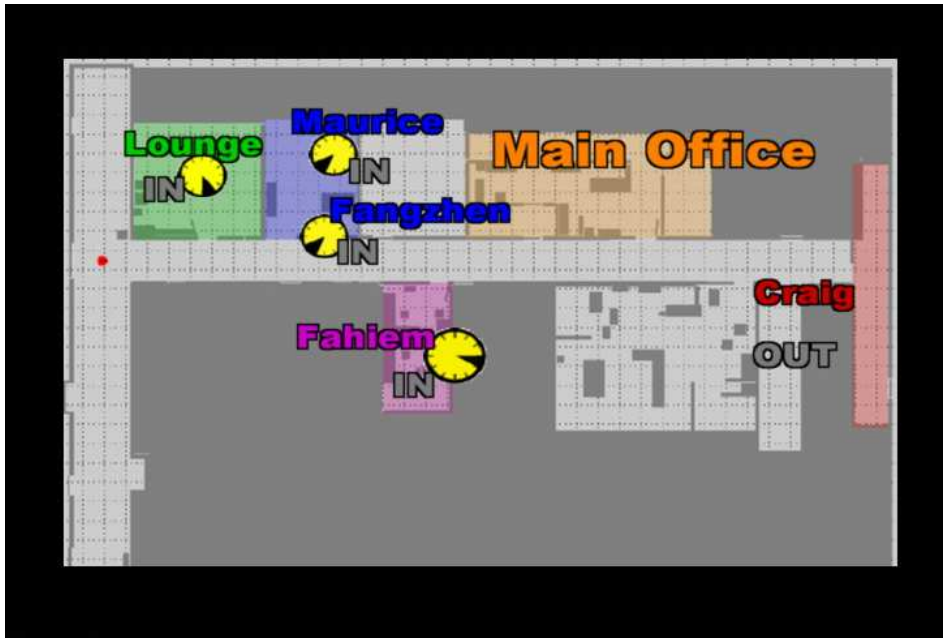
$$getOnFlight'_{Sketchy} \stackrel{\text{def}}{=}$$

$$\begin{array}{l}
\Sigma[(\pi a.a)^*; At(airport)?]; \\
\Sigma[(go(term1) \mid go(term2)); \\
\quad checkDepartures^*; \qquad /^* \text{ Sensing Action! } ^*/ \\
\quad (buy(magazine) \mid buy(newspaper)); \\
\quad \textbf{if } GateNo \geq 90 \textbf{ then } \{go(gate(GateNo)); buy(coffee)\} \\
\qquad\qquad\qquad\qquad \textbf{else } \{buy(coffee); go(gate(GateNo))\} \ ]; \\
boardPlane
\end{array}$$

Other work in the area

1. Execution monitoring: want to compare the results of executing GOLOG program to what was expected [de Giacomo et al 98]
2. Limited planning: would like to be able to introduce limited forms of planning as part of much larger GOLOG programs.
   use ideas of [Bacchus & Kabanza 96] for filtering
3. Knowledge/belief and knowledge-base programming:
   [Moore 85, Scherl & Levesque 92, Shapiro et al 00, Reiter 01]
4. Decision-theoretic GOLOG: DTGOLOG [Boutilier et al AAAI 2000]
5. Plan correctness: physical+epistemic feasibility [Sardina et al 04,06]

Golem running GOLOG



Golem

Outline

Conclusions

Up until very recently, "robot programming" was tackled mainly by engineers and hobbyists:

- required familiarity with workings of sensors and effectors, and
- programming languages very close to hardware.

(not unlike situation with first computers.)

Increasingly, because of:

- more stable robotic platforms, and
- better low-level interfaces and simulations,

it has become possible to consider robot programs that are much more portable and hardware independent.

Conclusions (cont.)

Situation calculus provides:

1. an expressive framework to reason about action and change;
2. several high-level agent programing languages.

Plus:

- √ everything with a clean semantics in classical logic!
- √ elaboration tolerant (many extensions in a parsimonious way).

Robotics by computer scientists!:

- abstraction, encapsulation, modularity;
- semantics / correctness, computability.